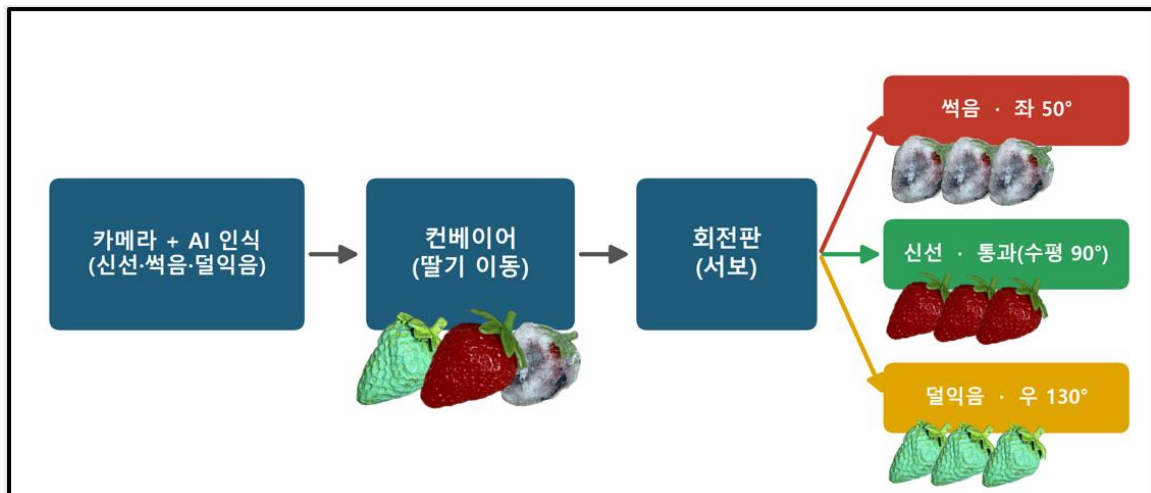


고급프로젝트 최종 보고서

딸기 자동 분류 시스템

AI 신선도 인식 기반 농산물 선별 자동화

TOPST SDV 3-Zone 임베디드 시스템 (AI-G · D3-G · VCP-G)



1 조

강요한 · 채지훈

중앙대학교 · 텔레칩스 팹리스 실무교육과정

0. 요약

딸기 자동 분류 시스템은 컨베이어 위를 지나는 딸기를 카메라와 AI(NPU)가 실시간으로 인식하여 신선·썩음·덜익음 3 종으로 판별하고, 끝단의 회전판이 종류별로 자동 분류하는 임베디드 시스템이다. 텔레칩스 자율주행 차량 키트의 인식 - 제어·통신 - 구동 구조에 착안하여, 칩과 운영체제, 프로그래밍 언어가 모두 다른 보드 세 개(AI-G·D3-G·VCP-G)를 이더넷·IPC·CAN 으로 연결하여 하나의 선별 라인으로 통합하였다.

AI 부문에서는 공개 데이터셋이 없는 한계를 모형 직접 제작·촬영으로 해결하여 539 장의 데이터셋을 구축하고, 경량 모델 YOLOv8n 을 학습하여 mAP50 약 0.99 를 달성하였다. ONNX 변환·INT8 양자화·컴파일 파이프라인으로 NPU 에 배포하는 과정에서 보드가 class 0 을 무시하는 버그를 발견하여 라벨 +1 shift 로 우회하였다. 제어 부문에서는 실제 환경에서 발생하는 오검출과 노이즈를 감지 박스·디바운스·gap 재무장·비차단 서보의 다단계 로직으로 보정하여 "딸기 1 개당 정확히 1 회 동작"을 구현하였으며, PC 에서 SSH 한 줄로 전체 시스템을 무선 기동·종료하는 운영 체계를 완성하여 세 보드의 풀체인 동작을 검증하였다.

1. 프로젝트 개요

1.1 배경 및 목표

농산물 선별은 신선도 판별과 물리적 분류가 모두 요구되는 노동집약적 공정이다. 본 프로젝트는 이 과정을 사람의 개입 없이 자동화하는 것을 목표로 하며, 나아가 딸기를 대상으로 스마트 팩토리형 선별 라인을 시험 제작하는 데 의의가 있다. 카메라 영상에서 AI 가 신선도를 판별하고, 기계가 종류별로 물리 분류하는 완결형 선별 라인을 임베디드 환경에서 구현하였다.

1.2 시스템 콘셉트

왼쪽에서 카메라와 AI 가 딸기를 인식하고, 컨베이어를 거쳐, 끝단의 회전판이 썩음은 좌(50 도), 덜익음은 우(130 도), 신선은 수평(90 도) 통과로 분류한다. 분류 결과는 LED 3 색과 LCD 누적 통계로 표시된다.

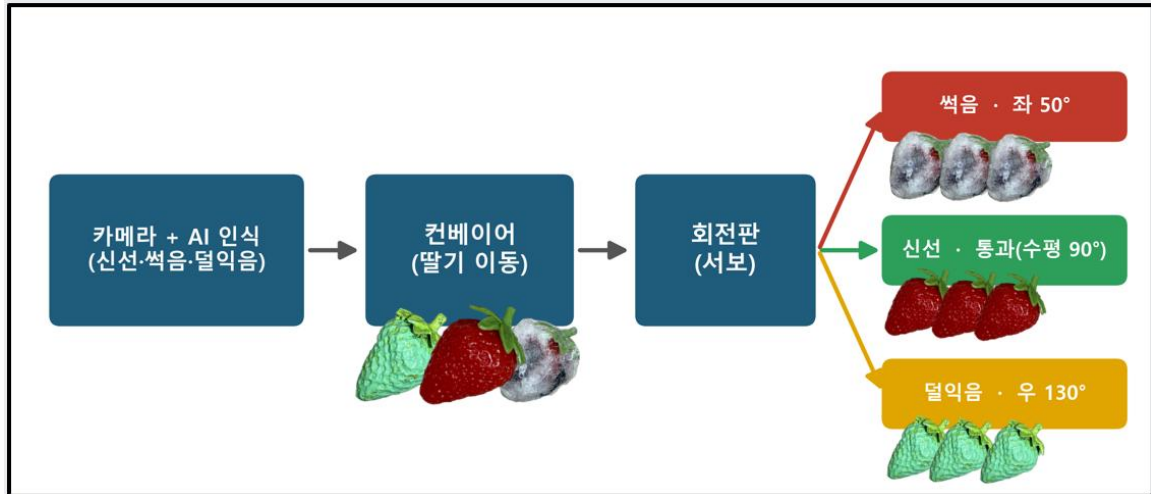


그림 1. 전체 동작 콘셉트 — 카메라·AI 인식, 컨베이어 이동, 회전판의 종류별 분류로 이어지는 흐름

1.3 역할 분담

담당	보드 / 영역	주요 작업
강요한	AI-G (인식)	데이터셋 구축, YOLOv8n 학습, NPU 변환·양자화·배포, 추론 연동, 발표
채지훈	D3-G · VCP-G (제어·구동), 시스템 통합	TCP 수신·분류 판단, IPC-CAN 브리지, 서보·LED·LCD 제어, 시스템 통합, 무선 운영, 실환경 튜, 통신 체인 연결, 발표 ppt, 대본 제작
공동	빠대 제작	3D 모델링·출력

2. 시스템 아키텍처

2.1 3-Zone 구성

역할이 다른 TOPST 교육 보드 세 개가 인식·제어·구동을 분담한다. 세 보드는 칩과 운영체제, 프로그래밍 언어가 모두 다르며, 이러한 이질적 보드를 하나로 통합하는 과정이 본 프로젝트의 핵심 난이도였다.

보드	역할	구성 및 핵심 기능
AI-G	인식 (Sensing)	카메라 + NPU(8 TOPS). YOLOv8n 추론 후 결과를 TCP 로 전송. Yocto Linux (Python 미탑재)
D3-G	제어 (Control)	A72(Linux)가 판단·트리거, R5(FreeRTOS)가 IPC-CAN 브리지. 듀얼코어
VCP-G	구동 (Actuator)	TCC70xx MCU. CAN 수신 후 회전판 서보·LED 3 색·LCD 제어

컨베이어 DC 모터는 어느 보드도 제어하지 않으며, 외부 전원과 컨트롤러로 별도 구동된다(속도 포함).

2.2 통신 체인과 데이터 흐름

카메라 인식 결과는 구간마다 다른 매체를 거쳐 액추에이터까지 전달된다. 이더넷(TCP) - IPC(공유메모리) - CAN 으로 이어지는 3 단 체인 구조이다.



그림 2. 통신 체인 — 인식(AI-G)에서 제어(D3-G)를 거쳐 구동(VCP-G)으로 이어지는 이더넷·IPC·CAN 데이터 흐름

구간	매체	형식 / 비고
AI-G - D3-G(A72)	이더넷 (TCP)	포트 5000. CLASS,CONF,X,Y 문자열 (bbox 중심 좌표)
A72 - R5	IPC 공유메모리	/dev/tcc_ipc_micom, snd 모드 패킷
R5 - VCP-G	CAN Bus	0x111 서보(각도), 0x114 LED, 0x116 LCD
컨베이어	외부 배선	보드 비관여 (외부 컨트롤러)

2.3 AI-G - D3-G 메시지 프로토콜 (TCP)

줄 단위 문자열을 포트 5000 으로 전송한다.

```
[좌 표]   썩 음   conf= 69%  X=511  Y=583
[좌 표]   썩 음   conf= 59%  X=496  Y=575
[좌 표]   썩 음   conf= 59%  X=486  Y=569
[좌 표]   썩 음   conf= 69%  X=481  Y=570
```

코드 1. AI-G - D3-G TCP 메시지 형식

필드	의미	비고
CLASS	프로젝트 클래스 (0~2)	0=신선, 1=썩음, 2=덜익음
CONF	신뢰도 (%)	MIN_CONFIDENCE 미만은 D3-G 에서 무시
X, Y	bbox 중심 좌표 (원본 px)	위치 트리거에 사용 (좌표가 곧 위치)
W, H	미사용	현재 0 고정

2.4 네트워크 구성

연결	IP / 포트	용도
AI-G eth	192.168.10.100	D3-G 와 이더넷 통신 (클라이언트)
D3-G eth	192.168.10.101 : 5000	AI-G - D3-G TCP 서버
D3-G (WiFi/USB)	192.168.0.x	PC 에서 SSH 원격 운영 / SCP

3. AI 학습 및 NPU 배포

담당: 강요한. 데이터셋 구축부터 학습, NPU 양자화, 보드 배포·추론까지를 다룬다. 전체 변환은 강의안의 YOLOv8 워크플로를 따르되 모델만 딸기 모델로 교체하였다. 변환 명령어 전문은 부록 A 에, 변환 과정에서의 트러블슈팅은 부록 C·D 에 정리하였다.

3.1 데이터셋 구축

딸기 신선도에 적합한 공개 데이터셋이 존재하지 않아, 신선·덜익음·썩음 모형을 직접 제작하고 다양한 거리와 각도에서 촬영한 뒤 Labellmg 로 라벨링하여 총 539 장(train 475, valid 64)의 데이터셋을 구축하였다. 촬영 환경 단위로 학습·검증 세트를 분리하여 데이터 누수를 방지하였다.

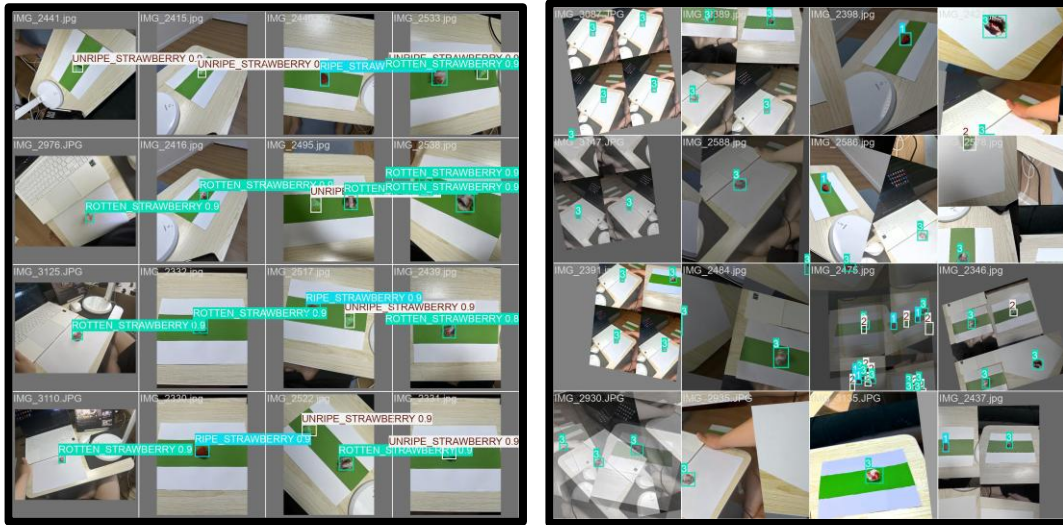


그림 3. 직접 촬영·라벨링한 딸기 데이터셋 예시 (객체 경계 상자 표시)

3.2 모델 학습

보드 메모리(2GB) 제약을 고려하여 경량 모델인 YOLOv8n(약 3M 파라미터)을 채택하였다. 약 6 회의 반복 학습과 mosaic-mixup 증강을 적용하여 최종 mAP50 약 0.99 를 달성하였다.

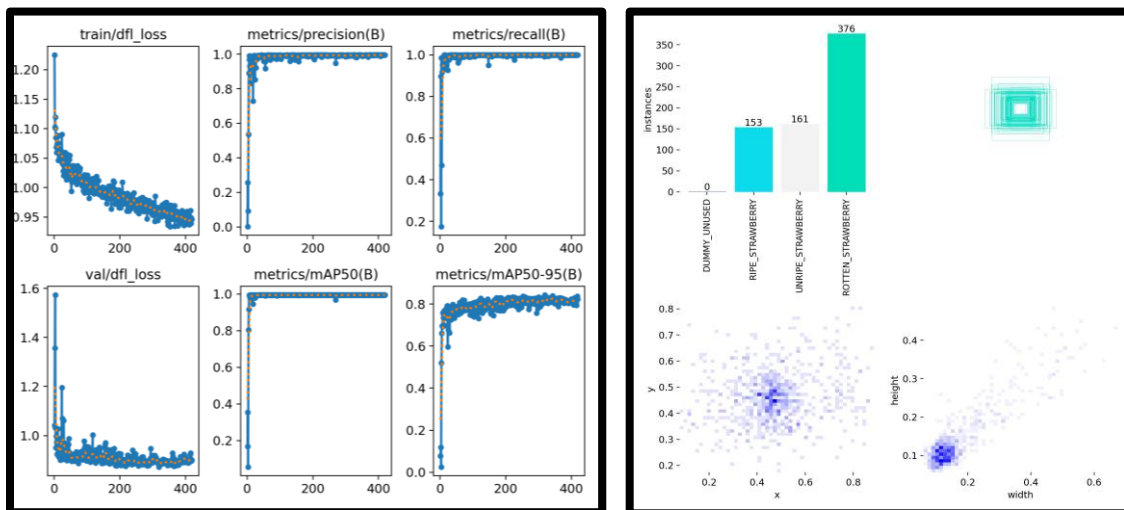


그림 4. 학습 결과 — 손실·정밀도·재현율·mAP 곡선 및 클래스 분포

3.3 NPU 변환 및 배포

YOLOv8 의 후처리(DFL, NMS, sigmoid)는 NPU 에서 비효율적이므로, 백본·넥·헤드의 raw 합성곱 출력까지만 NPU 에서 연산하고 후처리는 CPU 에서 수행하도록 분리하였다. 이후 FP32 변환과 INT8 양자화, 컴파일을 거쳐 보드에 배포하였으며, 변환 결과 NPU 호환 레이어가 100%(미지원 0 개)로 확인되었다. 보드에서는 tcnnapp 를 통해 약 2 FPS 로 추론한다.

3.4 핵심 발견 — NPU class 0 버그와 라벨 +1 Shift

PC 시뮬레이터에서는 모든 클래스가 정상적으로 검출되었으나, 보드에서만 class 0 이 검출되지 않는 현상이 발생하였다. 이를 해결하기 위해 학습 라벨 전체를 1 씩 이동시켜 class 0 을 비워두는(DUMMY) 방식으로 우회하고, 클래스 수를 4 로 지정하여 재학습·변환하였다.

그 결과 최종 tcnnapp 출력은 한 칸 더 밀려 class 2 가 신선, class 3 이 덜익음, class 4 가 썩음으로 나타나며, 제어부의 ai_result_sender.sh 가 이 값을 프로젝트 클래스로 매핑한다. 이 발견은 검증 환경(PC)과 실행 환경(보드)의 차이를 드러낸 사례이다.

tcnnapp 출력	의미	프로젝트 클래스 / 동작
class 2	fresh	신선 / 수평 90 도 통과
class 3	unripe	덜익음 / 우 130 도
class 4	rotten	썩음 / 좌 50 도
그 외	dummy 등	무시

4. 제어 및 시스템 통합

D3-G 가 AI 결과를 받아 분류를 판단하고, CAN 을 통해 VCP-G 의 회전판·LED·LCD 를 구동한다. 모든 분류 파라미터를 코드 상수로 분리하여 재플래싱 없이 즉시 조정할 수 있도록 설계하였다.

4.1 회전판 서보 — 분류 동작

분류 게이트는 SG90 위치제어 서보 1 개로 구현하였다. 초기에 사용한 연속회전형(MG90S)은 시간제어 방식이라 정지 위치가 일정하지 않아, 목표 각도로 정확히

멈추는 위치제어형(SG90)으로 교체하였다. 각도와 유지시간은 fruit_controller.py 상수로 분리되어 재실행만으로 튜닝된다.

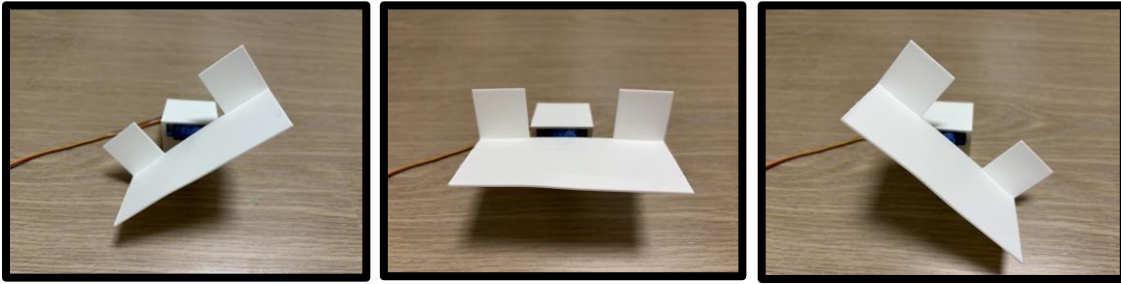


그림 5. 회전판(분류 게이트) — SG90 서보에 직결되어 좌·우·수평으로 기울어 종류별 배출

클래스	회전판 각도	동작	LED
신선	수평 90 도	유지 후 통과	초록
썩음	좌 50 도	기울임 후 1.3 초 뒤 90 도 복귀	빨강
덜익음	우 130 도	기울임 후 1.3 초 뒤 90 도 복귀	노랑

신호 사양(위치제어 SG90): 주기 20ms(50Hz), 듀티 = 0.5ms + 각도×(2ms/180 도).

0 도=0.5ms, 90 도=1.5ms, 180 도=2.5ms 이며, 위치제어 서보에서는 듀티가 곧 물리 각도에 대응한다.

4.2 CAN ID 체계 (D3-G - VCP-G)

CAN ID	기능	데이터 형식
0x111	회전판 서보	byte0 = 목표 각도(0~180 도), 1 바이트 (90 도=5A)
0x114	LED 제어	byte0 = LED_ID, byte1 = ON(01)/OFF(00)
0x116	LCD 통계	byte0~3 = total, fresh, rotten, unripe (4 바이트)


```
// 서보
sudo python3 IPC_Example.py snd --canID 0x111 --sndDataHex 5A # 서보1 → 90° (신선)
sudo python3 IPC_Example.py snd --canID 0x111 --sndDataHex 8C # 서보1 → 130° (덜익음)
sudo python3 IPC_Example.py snd --canID 0x111 --sndDataHex 32 # 서보1 → 50° (썩음)

//LED
sudo python3 IPC_Example.py snd --canID 0x114 --sndDataHex 0101 # 빨강 ON
sudo python3 IPC_Example.py snd --canID 0x114 --sndDataHex 0201 # 노랑 ON
sudo python3 IPC_Example.py snd --canID 0x114 --sndDataHex 0401 # 초록 ON
sudo python3 IPC_Example.py snd --canID 0x114 --sndDataHex 0000 # 전체 OFF

// LCD
sudo python3 IPC_Example.py snd --canID 0x116 --sndDataHex 05020201 # T:5 F:2 R:2 U:1
```

코드 2. IPC 를 통한 CAN 전송 예시 — 서보(0x111)·LED(0x114)·LCD(0x116) 명령

LED ID 매핑(0x114, byte0)은 0x00 ALL OFF, 0x01 RED(썩음), 0x02 YELLOW(덜익음), 0x04 GREEN(신선)이며, 바이트 길이를 엄수한다.

4.3 LED 및 LCD 표시

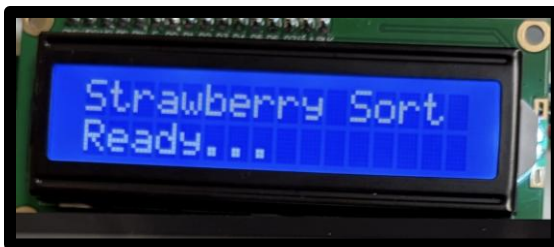
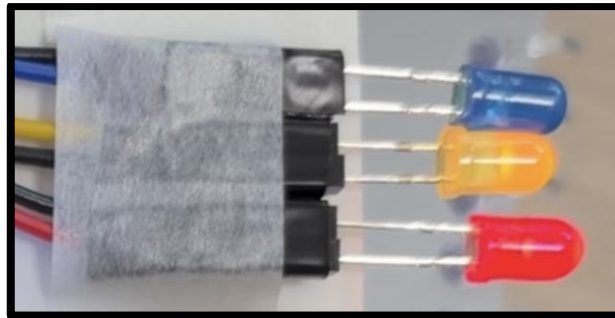


그림 6. 분류 결과 표시 — LED 3 색 점등과 LCD 대기·누적 통계 화면

분류 결과는 LED 3 색과 LCD 로 즉시 시각화된다. LED 는 썩음을 빨강, 덜익음을 노랑, 신선을 초록으로 표시하며, 분류와 동시에 점등된 뒤 약 1 초 후 소등된다. 색마다 독립 타이머로 동작하므로 여러 색이 동시에 점등될 수 있다. LED 제어는 회전판 모터 동작을 방해하지 않도록 논블로킹(백그라운드) 방식으로 처리하여 분류 동작에 항상 우선순위를

두었다. LCD 에는 분류 누적 통계(전체·신선·씩음·덜익음 개수)가 표시되며, 분류 세션이 종료(30 초 무감지 또는 종료 명령)되면 최종 통계를 보여준다.

4.4 분류 신뢰성 로직 — "딸기 1 개당 정확히 1 회"

카메라가 약 2 FPS 로 느리고 NPU 노이즈(좌표 점프 유령, 구석에 멈춘 유령, 오분류)가 많아, 단순히 "감지하면 바로 동작"하는 방식으로는 신뢰성이 부족하였다. fruit_controller.py 는 다음 파이프라인을 통해 한 딸기를 정확히 한 번만 분류한다.

단계 (상수)	역할
신뢰도 필터	MIN_CONFIDENCE=50 미만 무시
2D 박스 게이트	TRIGGER_X 230~375 AND TRIGGER_Y 390~490 — 회전판 자리만. 덜익음만 축소 박스(X_HI 335 / Y_HI 455)로 늦게 트리거
신선 우선	FRESH_VETO — 박스 안에서 신선이 1 프레임이라도 잡히면 그 딸기를 신선으로 간주, 2.0 초간 모터 차단
디바운스	TRIGGER_CONFIRM=3 — 연속 3 프레임 잡혀야 인정 (1~2 프레임 유령 무시)
gap 설정	COUNT_GAP=1.5 초 — 사라졌다 다시 와야 새 딸기로 인정 (중복 카운트 방지)
비차단 서보	_servo_lock(non-blocking) — 동작 중 들어온 트리거는 건너뛴 (백로그 방지)
LED / 종료	led_pulse 1.0 초 점등, end_watcher 30 초 무감지 시 LCD 통계 표시 후 종료



```

[좌 표] 덜 익 음  conf= 69%  X=259  Y=447
[좌 표] 덜 익 음  conf= 59%  X=257  Y=453
[좌 표] 덜 익 음  conf= 77%  X=256  Y=452
→ [덜 익 음 →오 른 쪽] 90°→130° (유 지 1.3s) →90°

[감 지] 덜 익 음 (신뢰도 77%, X=256 Y=452)
[통 계] 전 체 :3 | 신 선 :1(33%) | 썩 음 :1(33%) | 덜 익 음 :1(33%)

```

그림 7. 감지 박스(빨간 영역)와 이에 대응하는 D3-G 분류 로그 (좌표·신뢰도·누적 통계)

4.5 튜닝 상수 (재실행만으로 조정, 재플래싱 불필요)

상수	값	의미
SERVO_DEFAULT/LEFT/RIGHT_DEG	90 / 50 / 130	수평 / 썩음(좌) / 덜익음(우)
SERVO_HOLD_TIME	1.3	기울임 유지(초) 후 복귀
TRIGGER_X_LO~HI / Y_LO~HI	230~375 / 390~490	감지 박스 좌표
TRIGGER*_UNRIPE	X_HI 335 / Y_HI 455	덜익음 전용 축소 박스
FRESH_VETO_TIME	2.0	신선 보호 차단 시간(초)
TRIGGER_CONFIRM / COUNT_GAP	3 / 1.5	디바운스 / gap 재무장
MIN_CONFIDENCE / LED_ON_TIME	50 / 1.0	신뢰도 하한 / LED 점등(초)
END_SESSION_TIMEOUT	30	무감지 자동 종료(초)

96	SERVO_DEFAULT_DEG = 90	105	TRIGGER_Y_LO = 390
97	SERVO_LEFT_DEG = 50	106	TRIGGER_Y_HI = 490
98	SERVO_RIGHT_DEG = 130	107	TRIGGER_X_LO = 230
99	SERVO_HOLD_TIME = 1.3	108	TRIGGER_X_HI = 375

그림 8. 코드 상수로 분리한 튜닝 파라미터 (서보 각도·트리거 박스 좌표 등)

4.6 핀맵 (VCP-G firmware - 보드 헤더)

장치	종류	firmware 핀 / 채널	보드 위치
LED RED (썩음)	GPIO	GPA(6)	J8D104 핀 16
LED YELLOW (덜익음)	GPIO	GPA(28)	J8D104 핀 18
LED GREEN (신선)	GPIO	GPA(29)	J8D104 핀 19
회전판 서보 (SG90)	PDM	ch1, PERICH_CH1 - GPB(7)	J3D100 핀 1
LCD	I2C	CH3 / PORT7, ADDR 0x27	J10D100 GPC6/7

4.7 무선 통합 운영

PC 에서 D3-G 에 WiFi SSH 로 fruit_controller.py 한 줄을 실행하면 AI-G 인식이 자동
기동되고, 30 초 무감지 시 전체가 자동 종료된다. AI-G IP 영구화(systemd)와 무비밀번호
SSH 키 설정으로 PC 연결 케이블(UART, USB-C)을 모두 제거하고, WiFi SSH 만으로
콜드부팅부터 운영까지 동작함을 확인하였다.

```
# PC에서 D3-G에 WiFi SSH 접속 후, 한 줄로 풀체인 기동
sudo python3 /home/topst/fruit_controller.py
# [NET] eth0 고정 - [AI-G] 인식 시작 SSH - [좌표]/[감지] 출력
# 종료: Ctrl+C 또는 30초 무감지 (둘 다 LCD 최종 통계 표시)
```

코드 3. 무선 한 줄 운영 명령 — D3-G 가 AI-G 를 자동 지휘

5. 집중한 부분 — 실환경 문제와 해결

mAP 0.99 의 검증 점수에도 불구하고, 실제 환경에서는 카메라의 느린 속도와 다양한 각도,
노이즈로 인해 오검출이 발생하였다. AI 가 완전히 막을 수 없는 부분을 제어 로직으로
보정한 것이 본 프로젝트에서 가장 공들인 지점이다.

핵심은 세 가지 보정 로직이며, 각 코드의 역할은 다음과 같다. 첫째, 2D 감지 박스(코드 4)는
검출된 좌표가 회전판 직전의 지정 영역(X·Y 범위) 안에 들어올 때만 트리거하여 너무
이르거나 예민한 동작을 막는다. 둘째, 덜익음 전용 축소 박스(코드 5)는 과검출되는 덜익음
클래스에 한해 박스를 작게 잡아(오른쪽·아래 경계를 안쪽으로) 트리거 시점을 늦춰 다른
클래스와 타이밍을 맞춘다. 셋째, 신선 우선(코드 6)은 박스 안에서 신선이 한 번이라도
잡히면 일정 시간 서보와 카운트를 차단하여, 신선 딸기가 썩음·덜익음으로 오분류되어 잘못
배출되는 것을 방지한다.

105	TRIGGER_Y_LO = 390	# 박스 위 경계 (실측: 회전판 자리 딸기 Y≈390~490)
106	TRIGGER_Y_HI = 490	# 박스 아래 경계
107	TRIGGER_X_LO = 230	# 박스 왼쪽 X (실측: 진짜 딸기 X≈230~375, 구석유령 X=118은 제외됨)
108	TRIGGER_X_HI = 375	# 박스 오른쪽 X (오른쪽서 진입 → 넓힐수록 더 일찍 트리거=빠름)
356	in_box = (TRIGGER_X_LO <= cx <= x_hi) and (TRIGGER_Y_LO <= cy <= y_hi)	

코드 4. 2D 감지 박스 게이트 — 검출 좌표가 박스 영역(X·Y) 안일 때만 트리거

113	TRIGGER_X_HI_UNRIPE = 335	# 덜익음 전용 오른쪽 경계 (일반 375 → 40px 안쪽 = 더 늦게 트리거)
114	TRIGGER_Y_HI_UNRIPE = 455	# 덜익음 전용 아래 경계 (일반 490 → 35px 안쪽)
352	if class_id == CLASS_UNRIPE:	
353	x_hi, y_hi = TRIGGER_X_HI_UNRIPE, TRIGGER_Y_HI_UNRIPE	# 오른쪽·아래만 안쪽으로
354	else:	
355	x_hi, y_hi = TRIGGER_X_HI, TRIGGER_Y_HI	
356	in_box = (TRIGGER_X_LO <= cx <= x_hi) and (TRIGGER_Y_LO <= cy <= y_hi)	

코드 5. 덜익음 전용 축소 박스 — 클래스별로 트리거 영역을 다르게 적용

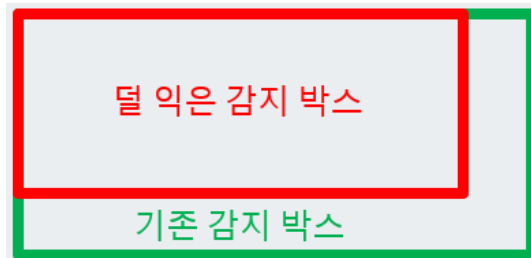


그림 9. 감지 박스 비교 — 기존 박스(초록)와 덜익음 전용 축소 박스(빨강)

123	FRESH_VETO = True	# False면 끄(원래대로 클래스별 독립 트리거)
124	FRESH_VETO_TIME = 2.0	# 신선 1회 감지 후 서보 차단 유지(초) ≈ 딸기 1개 박스 통과시간. 길수록 신선 보호↑
371	# — 신선 비토 설정: 박스에서 신선이 잡히면 잠깐 서보 차단(신선으로 간주) —	
372	if FRESH_VETO and class_id == CLASS_FRESH and in_box:	
373	if now >= _fresh_veto_until:	
374	print(f" [신선 비토] 신선 감지 → {FRESH_VETO_TIME:.1f}s 서보 차단(신선 통과)")	
375	_fresh_veto_until = now + FRESH_VETO_TIME	
376		
377	# — 신선 비토 중이면 썩음/덜익음 발동 안 함(이 딸기는 신선으로 간주 → 그냥 통과) —	
378	if (FRESH_VETO and class_id in (CLASS_ROTEN, CLASS_UNRIPE)	
379	and now < _fresh_veto_until):	
380	if in_box and _armed.get(class_id, True) and _inwin[class_id] >= TRIGGER_CONFIRM:	
381	_armed[class_id] = False	
382	print(f" [신선 비토] {name} 트리거 무시 → 신선으로 간주 (서보·카운트 x)")	
383	return # 서보/카운트/(잘못된)LED 모두 건너뛴	

코드 6. 신선 우선(FRESH_VETO) — 박스에서 신선이 잡히면 서보·카운트를 건너뛴

문제	원인	해결
너무 빠르고 예민한 감지	초기에는 Y 좌표만으로 판단	X·Y 2D 감지 박스 도입 — 회전판 자리에 왔을 때만 트리거
덜익음 과검출	덜익음 학습량이 많아 과검출	덜익음만 감지 박스를 축소 — 트리거를 늦춰 타이밍 정합
신선 오분류	신선이 가끔 썩음/덜익음으로 오검출	신선 우선(FRESH_VETO) — 신선이 1 회라도 잡히면 그 딸기는 신선으로 우선 처리

- 신선 우선의 근거: 모델은 신선을 썩음·덜익음으로 오분류하는 경우는 있으나, 반대로 썩음·덜익음을 신선으로 오분류하는 경우는 드물다. 따라서 신선 신호를 신뢰하여, 박스 안에서 신선이 한 번이라도 잡히면 그 딸기를 신선으로 간주하고 모터 동작을 차단함으로써 신선 딸기의 오배출을 방지하였다. 신선은 1 프레임, 썩음·덜익음은 3 프레임을 요구하도록 비대칭 설계하여 신선이 우선되도록 하였다.
- 튜닝의 코드화: 서보 각도와 유지시간, 감지 박스 좌표, 디바운스, gap 등 모든 값을 코드 상수로 분리하여 재플래싱 없이 즉시 조정할 수 있게 하였다. 이는 반복적인 현장 튜닝의 생산성을 크게 향상시켰으며, 본 시스템이 다양한 환경 변화에 빠르게 대응할 수 있는 기반이 되었다.

6. 결과물

6.1 3D 모델링

보드 수납 케이스와 분류 기구를 Python 파라메트릭 모델링으로 직접 설계하여 출력하였다(Bambu X1-Carbon).

- 컨트롤 타워: AI-G-D3-G-VCP 보드 3 개를 3 층으로 적층 수납하는 통합 하우징
- 회전판: SG90 에 직결되어 좌·우·수평으로 기울어 종류별로 배출
- 지지대: 회전판을 지면과 30 도 경사로 받쳐, 신선 딸기가 그대로 굴러 통과

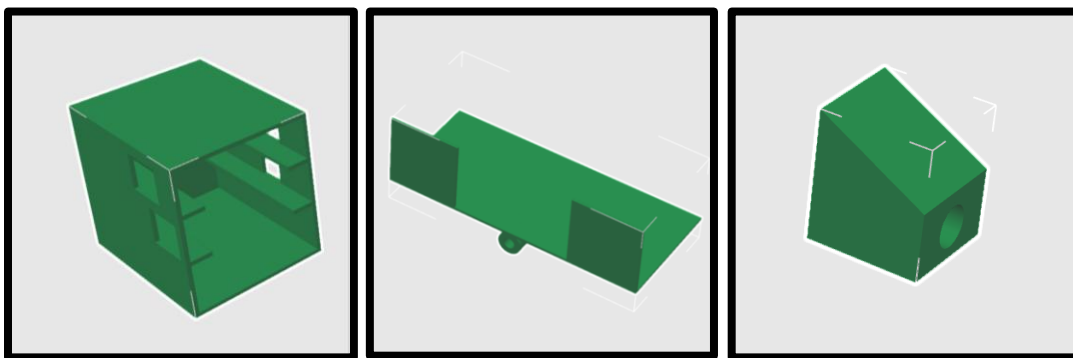


그림 10. 직접 모델링·출력한 기구물 — 컨트롤 타워, 회전판, 30 도 경사 지지대

6.2 통합 시스템

카메라·인식 보드에서 시작해 컨베이어를 거쳐 끝단의 회전판, 경사 지지대, 상태 표시 LED와 LCD까지 물리적으로 조립한 전체 시스템이다.

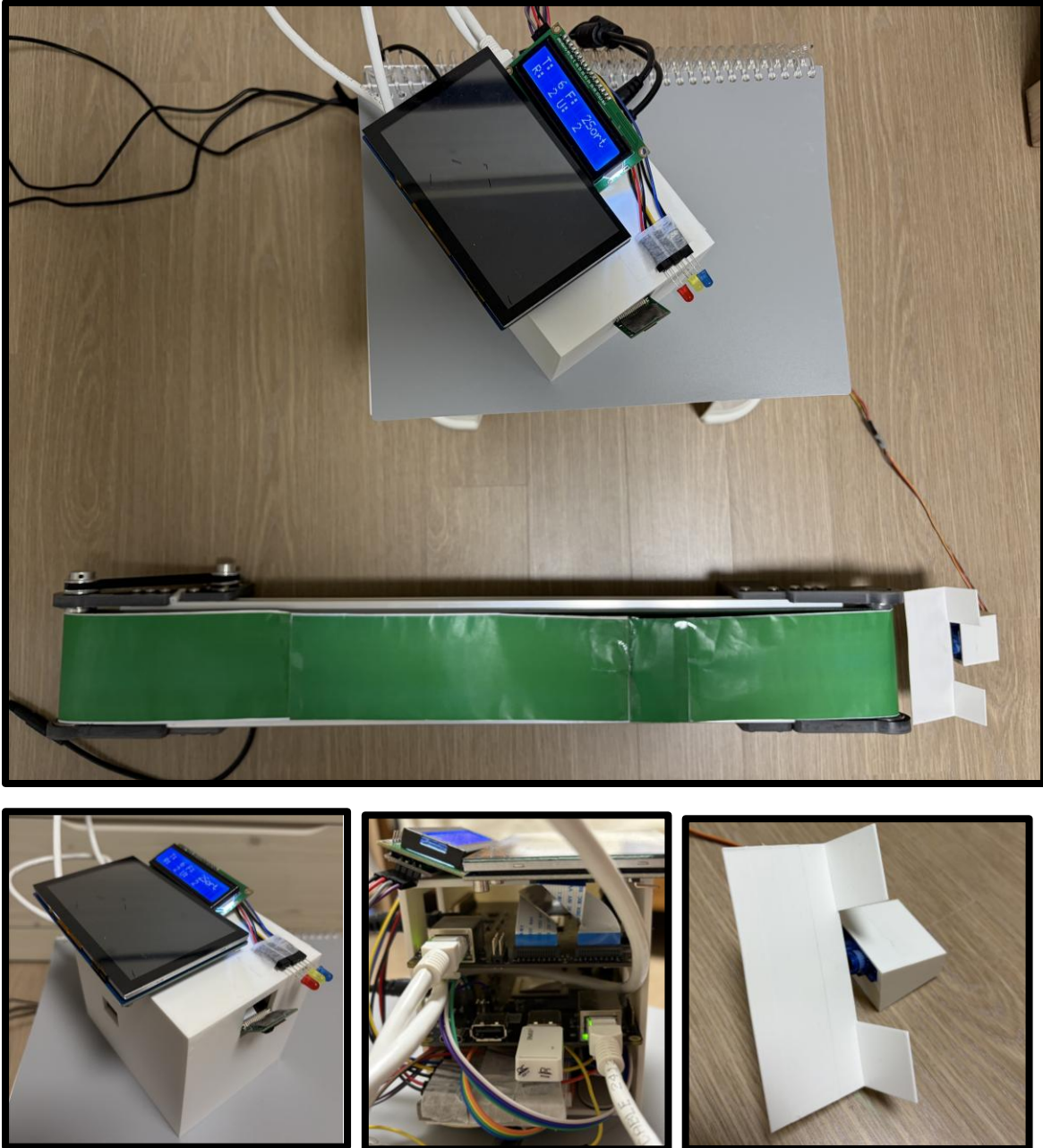


그림 11. 조립된 전체 시스템 — 컨베이어, 회전판, 컨트롤 타워, 상태 표시 LCD

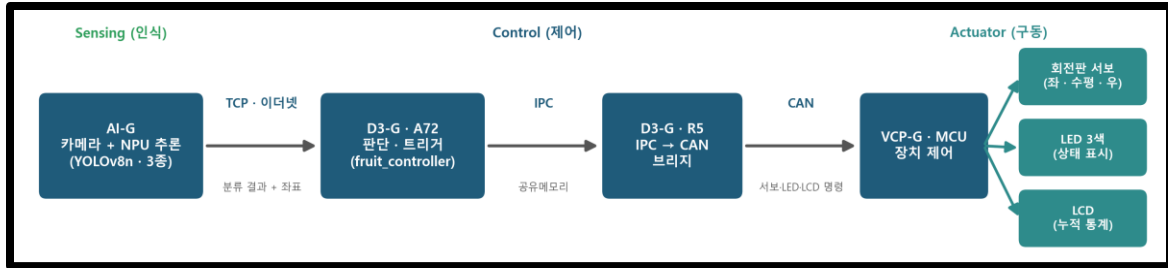


그림 12. 전체 처리 흐름 — 인식·제어·구동 보드 간 데이터 경로 요약

6.3 동작 검증

```

root@TOPST:/home/topst/Fabless_Education_blink/Step_0# sudo python3 /home/topst/fruit_controller.py
=====
말기 분류 - D3-G 제어부 (회전판 1서보 / 위치 트리거)
각도 : 신선 90° / 썩음 (원) 50° / 덜익음 (오) 130°, 유지 1.3s
트리거 박스 : X 230~375 AND Y 390~490 / 확인 3프레임 / 서보유지 1.3s
└ 덜익음 전용 (축소) : X 230~335 AND Y 390~455 (오른쪽 · 아래 줄여 늦게 트리거)
└ 신선 비토 ON: 박스에서 신선 1회 ↑ 감지 시 2.0s 서보 차단 (신선 보호)
※ 컨베이어는 외부 제어 — 이 코드는 관여 안 함
LED=감지 즉시 점등 (여러색 가능) · 종료시 OFF / LCD=종료시 결과 / 종료판정=30s 무감지
★오케스트레이션 ON: AI-G(192.168.10.100) SSH로 자동 시작/종료
=====
[NET] D3-G eth0 = 192.168.10.101 고정
[TCP] AI-G 연결 대기 중 ... (포트 5000)
[AI-G] 인식 시작 SSH → root@192.168.10.100
=====
말기 인식 결과 전송 시작
D3-G: 192.168.10.101:5000
=====
[TCP] AI-G 연결됨 : ('192.168.10.100', 52228)
[좌표] 썩음 conf= 69% X=511 Y=583
[좌표] 썩음 conf= 59% X=496 Y=575
[좌표] 썩음 conf= 59% X=486 Y=569
[좌표] 썩음 conf= 69% X=481 Y=570

[감지] 덜익음 (신뢰도 77%, X=256 Y=452)
[통계] 전체 :3 | 신선 :1(33%) | 썩음 :1(33%) | 덜익음 :1(33%)
[감지] 썩음 (신뢰도 83%, X=364 Y=490)
[통계] 전체 :5 | 신선 :1(20%) | 썩음 :2(40%) | 덜익음 :2(40%)
[감지] 신선 (신뢰도 69%, X=367 Y=467)
→ 신선 :수평(90°) 유지 → 통과
[통계] 전체 :6 | 신선 :2(33%) | 썩음 :2(33%) | 덜익음 :2(33%)

===== 분류 종료 (30s 무감지) → 결과 출력 =====
최종 : 전체 :6 | 신선 :2(33%) | 썩음 :2(33%) | 덜익음 :2(33%)
[AI-G] 인식 중지 SSH (pkill tcnnapp)
[TCP] AI-G 연결 끊김
[INFO] 종료됨
[종료] 30초 무감지 → 시스템 종료 . (다음 배치는 다시 실행)
  
```

그림 13. D3-G 보드 SSH 환경에서 전체 동작 과정 로그

AI-G 카메라 추론, D3-G 박스 트리거, VCP 서보·LED·LCD 까지 세 보드의 풀체인 동작을 확인하였으며, 실제로 딸기를 흘려보내 분류가 이루어짐을 검증하였다. LED 멀티 점등, LCD 통계 표시, 30 초 무감지 자동 종료, PC SSH 한 줄 무선 기동까지 모두 정상 동작함을 확인하였다.

7. 발전 가능성 및 결론

7.1 발전 가능성

항목	현재 구현	발전 방향
처리량	딸기가 하나씩 간격을 두고 이동	여러 개가 동시·밀집하여 와도 정확히 분류 (다중 객체 인식·제어 정교화)
속도	컨베이어 이동이 느림	고속·모션블러 대응 학습 + 빠른 트리거 타이밍
정확도	모형 딸기 539 장 학습	데이터 추가·클래스 밸런싱 + 실물 딸기 대량 테스트로 정량 검증
확장	3 종 분류	신선도 등급 세분화, 완전 무선화, 통계 대시보드

7.2 결론

본 프로젝트는 이질적인 임베디드 보드 세 개를 인식·제어·구동으로 분담·통합하여, AI 신선도 판별과 물리적 분류가 결합된 완결형 자동 선별 라인을 구현하였다. 공개 데이터가 없는 문제는 직접 데이터셋 구축으로, 보드 전용 NPU 버그는 라벨 shift 로, 실환경 오검출은 다단계 제어 로직으로 각각 해결하며 "학습된 모델이 실제 기구에서 신뢰성 있게 동작하기까지"의 전 과정을 경험하였다는 점에서 의의가 있다. 데이터와 속도, 확장성 측면의 과제를 보완한다면 실용적인 농산물 선별 자동화로 확장할 수 있으며, 궁극적으로 스마트 팩토리 시스템으로 발전시킬 수 있을 것으로 기대된다.

8. 사용 도구 및 참고자료

구분	도구 / 자료
교육과정	텔레칩스 팹리스 실무교육과정 (D01-D10) — TOPST 보드 환경, NPU 워크플로, GPIO-CAN-I2C
AI 학습	YOLOv8 (Ultralytics), PyTorch, Google Colab, Labellmg
NPU 배포	tc-nn-toolkit / Enlight SDK (ONNX - INT8 양자화 - 컴파일), tcnnapp
임베디드	Yocto Linux, FreeRTOS, WSL, FWDN(플래싱)
3D 모델링	Python 파라메트릭 모델링 (STEP/STL), Bambu Studio (X1-Carbon)

9. 멘토링 피드백

주요 평가. 현장 문제를 정의하고 후처리 보정으로 해결한 과정, 그리고 AI 의 한계를 비전·제어 알고리즘과 조합해 보완한 복합 시스템 설계 능력을 높이 평가받았으며, 데모도 성공적으로 시연되었다.

기술적 개선 제안.

- 1. 객체 추적(Object Tracking)** — 딸기마다 고유 ID 를 부여해 추적하면 중복 검출을 방지할 수 있다(하나의 ID 는 한 번만 처리).
- 2. 2-Stage 모델(Detection+Classification)** — YOLO 로 위치만 빠르게 찾아 크롭한 뒤 경량 분류 모델로 상태를 추론하여 속도와 분류 정확도를 함께 높인다.

현장 적용 인사이트. AI 는 환경 변화에 취약하므로, 현장 맞춤형 재학습과 복수 AI 모델 조합, 전통 비전·제어 알고리즘과의 융합이 필요하며, 본 프로젝트가 그 방향성을 잘 잡았다는 평을 받았다.

부록 A. 핵심 변환 명령어 (Converter · Quantizer · Compiler)

```
# Converter (.onnx -> .enlight, FP32)
python ./EnlightSDK/converter.py best_extracted.onnx \
    --type obj --add-detection-post-process best.bin \
    --dataset Custom --dataset-root <calibration_images> \
    --enable-track --mean 0 0 0 --std 1 1 1 \
    --num-class 4 --yolo-version v8 --enable-letterbox \
    --dfl-reg-max 16 --output-order cl \
    --logistic sigmoid --no-background      # class 0 버그 대응

# Quantizer (FP32 -> INT8)
python ./EnlightSDK/quantizer.py best.enlight --output best_quantized.enlight --m-std-4 7

# Compiler (NPU 명령어 + 후처리 생성)
python ./EnlightSDK/compiler.py best_quantized.enlight --th-iou 0.5 --th-conf 0.5
```

부록 B. 실행 런북 (운영 절차)

B.1 사전 준비

```
# AI-G IP 설정 (영구화 안 된 경우)
ifconfig eth0 192.168.10.100 netmask 255.255.255.0 up
# D3-G IP 고정 (NetworkManager 가 static IP 를 지우므로 managed 해제 후 설정)
sudo nmcli device set eth0 managed no
sudo ifconfig eth0 192.168.10.101 netmask 255.255.255.0 up
ping 192.168.10.101 -c 2      # AI-G 에서 연결 확인
ls /dev/video*               # 카메라 확인
```

B.2 실행 / 단독 테스트

```
# 1) D3-G - 제어 시작 (TCP 대기)
cd /home/topst && python3 fruit_controller.py
# 2) AI-G - 인식 시작
sh /home/root/ai_result_sender.sh

# TCP 단독 테스트 (스크립트 없이 AI-G 에서 수동 전송)
echo "CLASS:0,CONF:85,X:0.50,Y:0.50,W:0.30,H:0.30" | nc 192.168.10.101 5000
```

부록 C. 시스템 통합 디버깅 히스토리

프로젝트 진행 중 마주친 주요 이슈와 해결 과정이다. 다수가 실환경 특유의 문제(통신 IP 풀림, NPU 노이즈, I2C stuck 등)로, 소프트웨어와 하드웨어를 오가며 격리·해결하였다.

#	증상	원인 - 해결
1	IPC 전송해도 R5 무반응	R5 log on 미입력 (단, 콜드사이클 재검증 결과 현재 불필요)
2	서보 무반응 / 컨베이어 오동작	전송 형식 오류 - 서보 byte0=각도, 컨베이어 (action,speed)
3	LED 제어 안 됨	3 바이트 전송 - 2 바이트(id, on/off)로 변경
4	IPC sndrecv 크래시	TypeError(BufferedWriter) - snd 모드만 사용
5	python3: not found (AI-G)	Yocto 에 Python 없음 - sh + awk + nc 로 구현
6	No route to host / ping loss	eth0 IP 풀림 + NetworkManager 가 static IP flush - nmcli managed no + ifconfig 고정
7	트리거 방식 진화	쿨다운 - 엣지 트리거 - 2D 박스 + gap 으로 발전 (과다 카운트 해결)
8	NPU 유령 검출로 헛트리거	좌표 점프·구석 멈춘 유령 - 박스 + 디바운스(3 프레임)로 완화 (근본은 모델 품질)
9	밀려서 출력 / 빈 벨트에 모터 돔	서보 blocking 으로 동작 큐 백로그 - non-blocking(바쁘면 건너뛴)으로 변경
10	신선 딸기에 모터가 돔	모델이 그 프레임을 씹음/덜익음으로 오분류 - MIN_CONFIDENCE, CONFIRM 상향 (근본은 모델)
11	무선 run 에서 모터만 안 돔	점퍼 접합부 절연테이프로 접촉 불량 (LED-CAN 정상으로 격리) - 테이프 제거
12	run 끝나도 LCD 통계 안 뜸	update_lcd 가 종료 경로에서만 호출 - main() finally 에도 추가 (Ctrl+C 포함 표시)